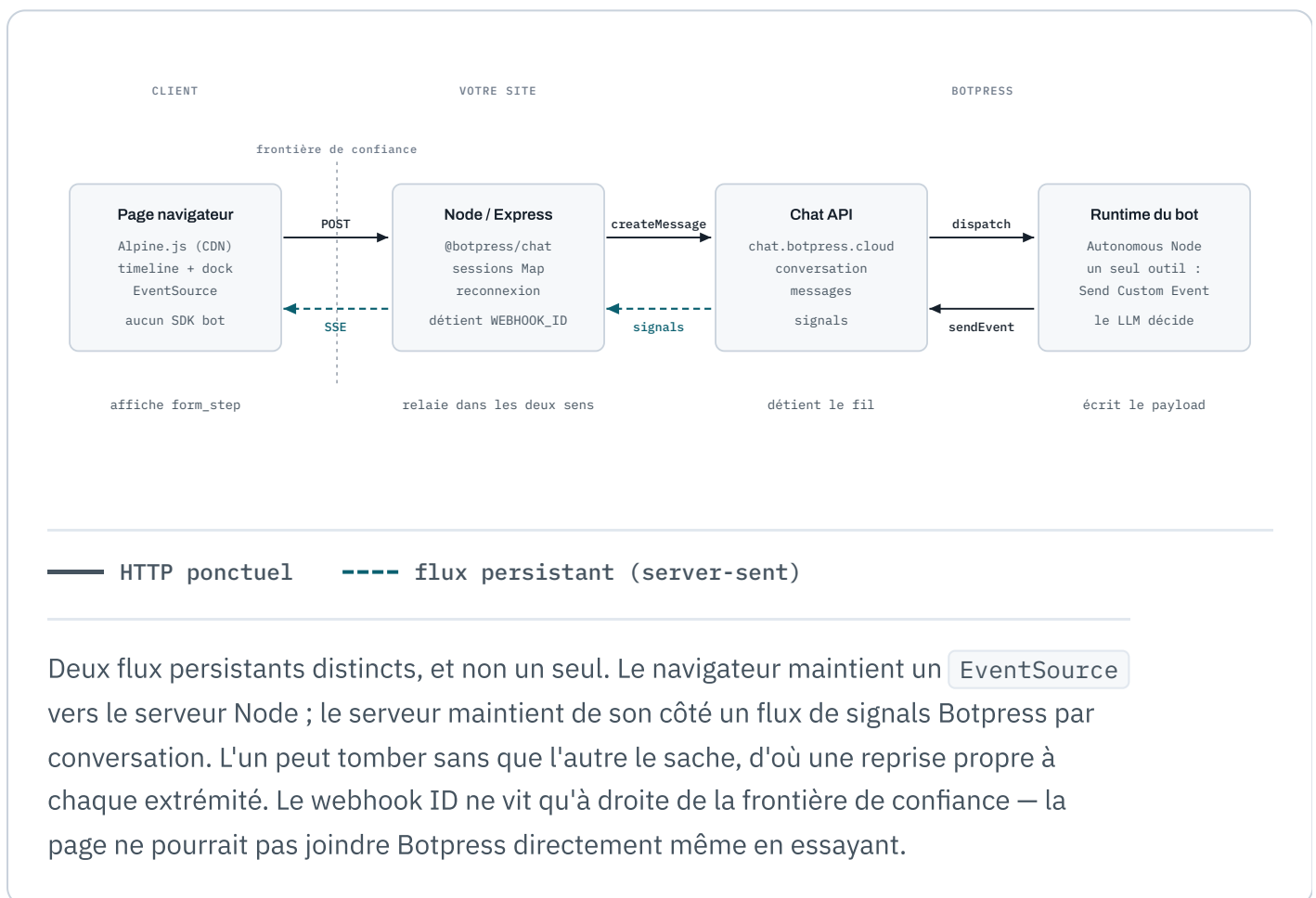


Formulaire de contact hybride sur la Chat API Botpress

Une page centrée sur le formulaire, où un LLM décide de la question suivante et rédige la réponse. Botpress fournit la conversation et l'intelligence ; chaque pixel reste le front-end du client. Voici comment les quatre parties dialoguent.

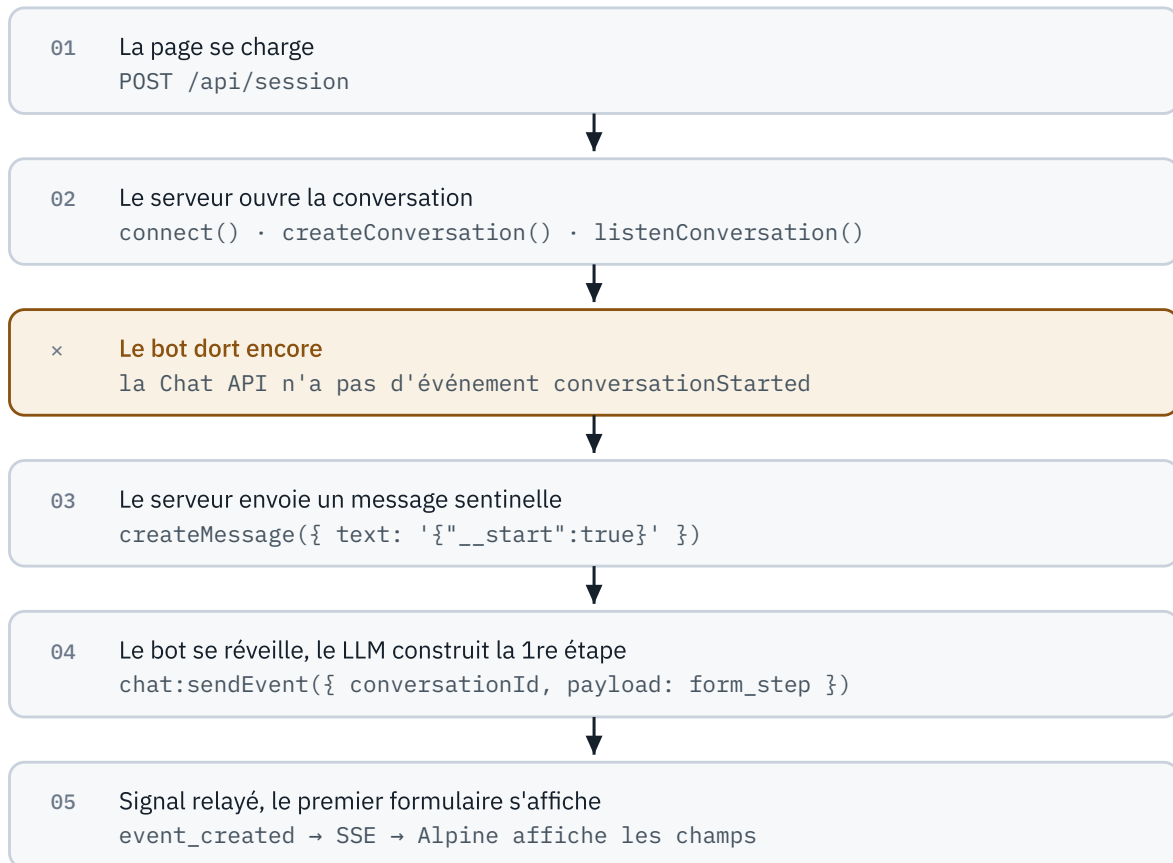
Les quatre parties

Rien ici n'est un widget de chat. La page possède sa propre interface et dialogue avec un petit serveur Node, seul à détenir le webhook ID et seul à parler à Botpress.



Réveiller le bot

La partie la moins évidente de la conception, et celle qui piègera quiconque la reconstruira. Créer une conversation ne la démarre pas.



Chaque tour suivant répète les étapes 03 à 05 : la page envoie les réponses du client sous forme de message, le bot répond par un nouveau `form_step`. Le message sentinelle n'existe que parce que l'étape 02 laisse le bot inactif — sans événement `conversationStarted` sur la Chat API, une conversation sans message n'atteint jamais le bot.

Ce qui circule

Un seul type de message dans chaque sens porte toute l'interaction. Le payload du bot est l'instruction d'affichage de la page ; celui de la page est du JSON brut dans un corps de message.

```
{
  "type": "form_step",
  "title": "Notre réponse",
  "done": false,
  "fields": [ ... ],      // formulaire à
afficher
  "answer": "...",        // réponse
rédigée
  "links": [ ... ],        // liste blanche
uniquement
  "choices": [ ... ]      // résoudre /
escalader
}
```

```
// réponses au formulaire courant
{ "name": "...", "topic": "Annulation" }

// complément en texte libre
{ "followUp": "..." }

// décision du client
{ "choice": "resolved" | "human" }
```

Endpoints du serveur

Quatre routes, toutes liées à une session. Le serveur est un relais doté d'un cycle de vie, pas une application.

ROUTE	RÔLE	APPEL BOTPRESS
POST /api/session	Ouvre une conversation, démarre le listener de signals, envoie le sentinelle	connect · createConversation · listenConversation · createMessage
GET /api/stream/:id	SSE vers le navigateur ; relaie les messages du bot et les événements <code>form_step</code>	– (relais seul)
POST /api/answers/:id	Envoie les réponses, un complément ou un choix en un seul message	createMessage
POST /api/session/:id/end	Détruit la session au redémarrage	listener.disconnect

Les points porteurs

Chacun a été découvert en le cassant. Ils ressemblent à des détails et se comportent comme des exigences.

Le Conversation ID doit être en Manual

L'outil Send Custom Event place son Conversation ID sur *Let AI Decide* par défaut. Ainsi, le LLM émet `conversation.id`, indéfini dans le sandbox, et chaque appel échoue. Passez-le en Manual avec `{{event.conversationId}}`.

Les deux flux ont besoin d'une reprise

Le `SignalListener` de Botpress a un watchdog de 60 secondes et aucune reprise propre — or une personne qui remplit un formulaire reste silencieuse bien plus longtemps. Le serveur se reconnecte ; les sessions sans navigateur attaché sont détruites après deux minutes.

La liste blanche de liens

Le bot ne peut émettre que des URL figurant dans une liste fixe de ses instructions, et jamais une URL brute dans le texte de la réponse. Sans cela, il invente des chemins plausibles.

L'escalade appartient au client

Il est interdit au bot d'annoncer qu'une demande est transférée ou qu'un conseiller confirmera. Anticiper le second bouton rend la réponse rédigée inutile — c'est la seule chose que le client peut déjà faire sans lui.

Preuve de concept — pas de production. L'identité repose sur une authentification anonyme automatique plutôt que sur les clés de session membre, les sessions vivent en mémoire du processus, et la reprise Zendesk n'est pas câblée. Les identifiants de bot et de webhook sont volontairement omis de cette page.